

```

/**
 * components/analytics/OperationsMode.tsx
 *
 * Changes from original:
 *
 * [Rec #4] Three-state anomaly acknowledgement: unacknowledged → watching → ack
 *          "Watching" prevents the binary ack-to-clear behaviour that causes al
 *          Operators mark something as "Watching" to say "I know, I'm monitorin
 *          it stays visible but no longer demands action.
 *
 * [Rec #5] Per-sensor drift thresholds. The original flagged any sensor deviat
 *          from baseline globally. Fast-varying sensors (fuel flow, RPM under 1
 *          fire constant false positives. Slow sensors (static temp, pressure)
 *          tighter windows. Threshold map is passed per sensorType.
 *
 * [Rec #10] DataFreshnessBadge on every prediction card. A failureRisk: 87% badg
 *          made from 6-hour-old telemetry looks identical to one made 30 second
 *
 * [Bug fix] getSeverityColor/getSeverityVariant now imported from shared lib/sev
 *          instead of duplicated locally.
 *
 * [Bug fix] toggleMutation now has an onError handler with a toast.
 */

import { useState, useCallback } from 'react';
import {
  Wifi, WifiOff, AlertTriangle, Check, Eye, TrendingUp,
  Brain, LineChart as LineChartIcon, RefreshCw,
} from 'lucide-react';
import { Link, useLocation } from 'wouter';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { Badge } from '@components/ui/badge';
import { Button } from '@components/ui/button';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@
import { ScenarioBanner } from './ScenarioBanner';
import { CollapsibleSection } from './CollapsibleSection';
import { ContextHelp } from './ContextHelp';
import { ResponsiveContainer, LineChart, Line, XAxis, YAxis, CartesianGrid, Toolt
import { formatDistanceToNow } from 'date-fns';
import { useOperationsModeData } from '@features/analytics';
import { useTelemetryStreams, SensorSparklineChart } from '@features/telemetry';
import { useQuery } from '@tanstack/react-query';
import { useIntersectionObserver } from '@hooks/useIntersectionObserver';

```

```
// New shared utilities
import { getSeverityVariant, type AnomalyAckStatus } from '@lib/severity';
import { DataFreshnessBadge } from '@components/ui/data-freshness-badge';
import { ConfidenceBadge } from '@components/ui/confidence-badge';
```

```
// — Per-sensor drift threshold map —————
//
// Recommendation #5 – replaces the global 15% constant.
// Sensors that naturally vary under load get wider windows;
// sensors on static equipment get tighter windows.
//
// Extend this map as your sensor catalogue grows.
```

```
const SENSOR_DRIFT_THRESHOLDS: Record<string, number> = {
  // High-variance sensors – normal operational range is wide
  rpm: 25,
  fuel_flow: 30,
  shaft_torque: 25,
  propeller_pitch: 20,
  exhaust_temp: 20,

  // Medium-variance sensors
  coolant_temp: 15,
  lube_oil_temp: 15,
  turbo_boost: 15,
  charge_air_temp: 15,

  // Low-variance sensors – tight tolerance, drift is significant
  bearing_temp: 10,
  vibration: 10,
  lube_oil_pressure: 8,
  fresh_water_temp: 8,

  // Default fallback for any sensor not listed
  _default: 15,
};
```

```
function getDriftThreshold(sensorType: string): number {
  // Normalise to lower_snake_case
  const key = sensorType.toLowerCase().replace(/[\s-]/g, '_');
  return SENSOR_DRIFT_THRESHOLDS[key] ?? SENSOR_DRIFT_THRESHOLDS._default;
}
```

```
// — Three-state anomaly acknowledgement —————
//
// Recommendation #4
```

```

interface AnomalyWithStatus {
  id?: string;
  equipmentId: string;
  equipmentName?: string;
  sensorType: string;
  value: number;
  unit?: string;
  zscore?: number;
  timestamp: string;
  ackStatus?: AnomalyAckStatus;
}

interface AnomalyRowProps {
  anomaly: AnomalyWithStatus;
  idx: number;
  ackStatus: AnomalyAckStatus;
  onWatch: (id: string) => void;
  onAcknowledge: (id: string) => void;
}

function AnomalyRow({ anomaly, idx, ackStatus, onWatch, onAcknowledge }: AnomalyR
  const anomalyId = anomaly.id ?? `${anomaly.equipmentId}-${anomaly.timestamp}`;

  const statusConfig: Record<AnomalyAckStatus, {
    badge: string;
    badgeClass: string;
    showWatch: boolean;
    showAck: boolean;
  }> = {
    unacknowledged: {
      badge: 'New',
      badgeClass: 'bg-red-100 text-red-700 dark:bg-red-950 dark:text-red-400',
      showWatch: true,
      showAck: true,
    },
    watching: {
      badge: 'Watching',
      badgeClass: 'bg-amber-100 text-amber-700 dark:bg-amber-950 dark:text-amber-
      showWatch: false,
      showAck: true,
    },
    acknowledged: {
      badge: 'Acknowledged',
      badgeClass: 'bg-muted text-muted-foreground',
      showWatch: false,
      showAck: false,
    },
  },

```

```

};

const cfg = statusConfig[ackStatus];

return (
  <div
    className=[
      'flex items-center justify-between gap-3 p-3 border rounded-lg transition
      ackStatus === 'acknowledged' ? 'opacity-50' : '',
    ].join(' ')
    data-testid={`item-anomaly-${idx}`}
  >
    <div className="flex-1 min-w-0">
      <div className="flex items-center gap-2 flex-wrap mb-0.5">
        <p className="font-medium text-sm" data-testid={`text-anomaly-equipment
          {anomaly.equipmentName ?? anomaly.equipmentId}
        </p>
        <span className={`text-xs px-1.5 py-0.5 rounded-full font-medium ${cfg.
          {cfg.badge}
        </span>
        { /* Data freshness on each anomaly - Rec #10 */ }
        <DataFreshnessBadge timestamp={anomaly.timestamp} staleThresholdMinutes
      </div>
      <p className="text-xs text-muted-foreground" data-testid={`text-anomaly-d
        {anomaly.sensorType}: {anomaly.value}{anomaly.unit} (
        {anomaly.zscore?.toFixed(1)}σ deviation
        {' · '}threshold ±{getDriftThreshold(anomaly.sensorType)}%)
      </p>
      <Badge variant="outline" className="mt-1 text-xs" data-testid={`badge-ano
        {formatDistanceToNow(new Date(anomaly.timestamp), { addSuffix: true })}
      </Badge>
    </div>

    <div className="flex gap-1.5 shrink-0">
      { /* View graph */ }
      <Link href={` /active-telemetry?equipment=${anomaly.equipmentId}&sensor=${
        <Button size="sm" variant="outline" data-testid={`button-view-graph-${i
          <LineChartIcon className="h-3 w-3 mr-1" />
          Graph
        </Button>
      </Link>

      { /* Watch - new third state */ }
      {cfg.showWatch && (
        <Button
          size="sm"
          variant="outline"

```

```

        onClick={() => onWatch(anomalyId)}
        title="Mark as Watching - I'm aware and monitoring"
        data-testid={`button-watch-anomaly-${idx}`}
      >
        <Eye className="h-3 w-3 mr-1" />
        Watch
      </Button>
    )}

    { /* Acknowledge */ }
    {cfg.showAck && (
      <Button
        size="sm"
        variant="outline"
        onClick={() => onAcknowledge(anomalyId)}
        data-testid={`button-acknowledge-anomaly-${idx}`}
      >
        <Check className="h-3 w-3 mr-1" />
        Ack
      </Button>
    )}
  </div>
</div>
);
}

// — Main component —————

export function OperationsMode() {
  const [, navigate] = useLocation();
  const [selectedVessel, setSelectedVessel] = useState<string>('all');
  const [selectedEquipment, setSelectedEquipment] = useState<string>('all');

  // Three-state ack map - Rec #4
  // Maps anomalyId → AnomalyAckStatus
  const [ackStatuses, setAckStatuses] = useState<Map<string, AnomalyAckStatus>>(<

  const handleWatch = useCallback((id: string) => {
    setAckStatuses(prev => new Map(prev).set(id, 'watching'));
  }, []);

  const handleAcknowledge = useCallback((id: string) => {
    setAckStatuses(prev => new Map(prev).set(id, 'acknowledged'));
  }, []);

  const {
    isConnected,

```

```

    latestTelemetry,
    failurePredictions,
    criticalEquipment,
    warningEquipment,
    healthyEquipment,
    driftingSensors,
    highConfidencePredictions,
    avgPredictionConfidence,
    equipmentHealthTrends,
    unacknowledgedAnomalies,
  } = useOperationsModeData();

const { data: vessels = [] } = useQuery<Array<{ id: string; name: string }>>({
  queryKey: ['/api/vessels'],
});

const { data: equipment = [] } = useQuery<Array<{ id: string; name: string; ves
  queryKey: ['/api/equipment'],
});

const filteredEquipment = selectedVessel === 'all'
  ? equipment
  : equipment.filter(e => e.vesselId === selectedVessel);

const [telemetrySectionRef, isTelemetrySectionVisible] = useIntersectionObserve
  rootMargin: '100px',
  triggerOnce: true,
});

const { streams, isLoading: streamsLoading, refetch: refetchStreams } = useTele
  vesselId: selectedVessel === 'all' ? undefined : selectedVessel,
  equipmentId: selectedEquipment === 'all' ? undefined : selectedEquipment,
  hours: 1,
  refreshInterval: 30_000,
  enabled: isTelemetrySectionVisible,
});

const handleViewDetails = useCallback((equipmentId: string, sensorType: string)
  navigate(`/active-telemetry?equipment=${equipmentId}&sensor=${sensorType}`);
}, [navigate]);

// Split anomalies into three buckets based on ack status
const anomaliesWithStatus = (unacknowledgedAnomalies ?? []).map(a => ({
  ...a,
  ackStatus: ackStatuses.get(a.id ?? `${a.equipmentId}-${a.timestamp}`) ?? 'una
}));

```

```

const newAnomalies      = anomaliesWithStatus.filter(a => a.ackStatus === 'unac
const watchingAnomalies = anomaliesWithStatus.filter(a => a.ackStatus === 'watc
const acknowledgedCount = anomaliesWithStatus.filter(a => a.ackStatus === 'ackn

// Per-sensor drift: filter drifting sensors against their individual threshold
const actuallyDriftingSensors = (driftingSensors ?? []).filter(sensor => {
  const threshold = getDriftThreshold(sensor.sensorType);
  return (sensor.deviationPct ?? 15) > threshold;
});

return (
  <div className="space-y-6">
    <ScenarioBanner
      type="info"
      title="Operations Mode – Real-Time Monitoring"
      description="Monitor live equipment health, telemetry streams, and operat
    />

    { /* Connection status */ }
    <Card data-testid="card-connection-status">
      <CardHeader className="pb-3">
        <div className="flex items-center justify-between">
          <CardTitle className="text-base">Live Connection Status</CardTitle>
          <div className="flex items-center gap-2" data-testid="connection-indi
            { isConnected ? (
              <>
                <Wifi className="h-4 w-4 text-green-500" data-testid="icon-conn
                  <span className="text-sm text-green-500" data-testid="status-co
                </span>
              </>
            ) : (
              <>
                <WifiOff className="h-4 w-4 text-red-500" data-testid="icon-dis
                  <span className="text-sm text-red-500" data-testid="status-conn
                </span>
              </>
            ) }
          </div>
        </div>
      </CardHeader>
      <CardContent>
        { latestTelemetry && (
          <div className="flex items-center gap-2" data-testid="latest-telemetr
            <p className="text-sm text-muted-foreground" data-testid="text-late
              Latest: { latestTelemetry.sensorType } = { latestTelemetry.value } { la
                { ' ' } } ( { formatDistanceToNow(new Date(latestTelemetry.timestamp), {
              </p>
              { /* Rec #10 – data freshness on live reading */ }
            <DataFreshnessBadge

```

```

        timestamp={latestTelemetry.timestamp}
        staleThresholdMinutes={5}
        criticalThresholdMinutes={15}
      />
    </div>
  )}
</CardContent>
</Card>

{/* Prediction status + drift */}
<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  <Card data-testid="card-prediction-status">
    <CardHeader className="pb-3">
      <CardTitle className="text-sm flex items-center gap-2">
        <Brain className="h-4 w-4" />
        AI Prediction Status
      </CardTitle>
    </CardHeader>
    <CardContent>
      <div className="space-y-2">
        <div className="flex justify-between items-center" data-testid="met">
          <span className="text-sm text-muted-foreground">Avg Confidence</span>
          { /* Rec #1 – educated confidence badge */ }
          <ConfidenceBadge confidence={avgPredictionConfidence} showPercent />
        </div>
        <div className="flex justify-between items-center" data-testid="met">
          <span className="text-sm text-muted-foreground">High Confidence</span>
          <Badge variant="default" className="min-w-[3rem] justify-center">
            {highConfidencePredictions.length}
          </Badge>
        </div>
        <div className="flex justify-between items-center" data-testid="met">
          <span className="text-sm text-muted-foreground">Active Prediction</span>
          <Badge variant="outline" className="min-w-[3rem] justify-center">
            {failurePredictions.length}
          </Badge>
        </div>
      </div>
    </CardContent>
  </Card>

  { /* Rec #5 – only show drift card if sensors actually exceed their indivi
  {actuallyDriftingSensors.length > 0 && (
    <Card className="border-amber-500" data-testid="card-telemetry-drift">
      <CardHeader className="pb-3">
        <CardTitle className="text-sm flex items-center gap-2 text-amber-60">
          <TrendingUp className="h-4 w-4" />

```



```

        Telemetry Drift Detected
    </CardTitle>
</CardHeader>
<CardContent>
    <div className="space-y-2">
        <div className="flex justify-between items-center" data-testid="m
            <span className="text-sm text-muted-foreground">Sensors exceedi
            <span className="text-lg font-bold text-amber-600" data-testid=
                {actuallyDriftingSensors.length}
            </span>
        </div>
        <p className="text-xs text-muted-foreground" data-testid="text-dr
            Deviating beyond per-sensor tolerance – review calibration
        </p>
        <div className="mt-2 space-y-1" data-testid="list-drifting-sensor
            {actuallyDriftingSensors.slice(0, 3).map((sensor, idx) => (
                <div key={`drift-${sensor.equipmentId}-${sensor.sensorType}`}
                    <span>
                        <span className="font-medium" data-testid={`text-drift-se
                            <span className="text-muted-foreground"> on {sensor.equip
                        </span>
                        <span className="text-amber-600 font-medium">
                            threshold:  $\pm$ {getDriftThreshold(sensor.sensorType)}%
                        </span>
                    </div>
                </div>
            )))
        </div>
    </CardContent>
</Card>
    )}
</div>

{/* Equipment health counts */}
<div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
    <Card className="border-red-500/50" data-testid="card-critical-equipment"
        <CardHeader className="pb-2">
            <div className="flex items-center justify-between">
                <CardTitle className="text-sm font-medium">Critical</CardTitle>
                <ContextHelp title="Critical Equipment" description="Equipment with
            </div>
        </CardHeader>
        <CardContent>
            <div className="text-3xl font-bold text-destructive" data-testid="tex
                {criticalEquipment.length}
            </div>
        </CardContent>
    </div>

```

```

</Card>
<Card className="border-amber-500/50" data-testid="card-warning-equipment"
  <CardHeader className="pb-2">
    <div className="flex items-center justify-between">
      <CardTitle className="text-sm font-medium">Warning</CardTitle>
      <ContextHelp title="Warning Equipment" description="Equipment with
    </div>
    </CardHeader>
    <CardContent>
      <div className="text-3xl font-bold text-amber-600" data-testid="text-
        {warningEquipment.length}
      </div>
    </CardContent>
  </Card>
<Card className="border-green-500/50" data-testid="card-healthy-equipment"
  <CardHeader className="pb-2">
    <div className="flex items-center justify-between">
      <CardTitle className="text-sm font-medium">Healthy</CardTitle>
      <ContextHelp title="Healthy Equipment" description="Equipment with
    </div>
    </CardHeader>
    <CardContent>
      <div className="text-3xl font-bold text-green-600" data-testid="text-
        {healthyEquipment.length}
      </div>
    </CardContent>
  </Card>
</div>

{/* Critical equipment list */}
{criticalEquipment.length > 0 && (
  <Card className="border-destructive" data-testid="card-critical-list">
    <CardHeader>
      <div className="flex items-center gap-2">
        <AlertTriangle className="h-5 w-5 text-destructive" />
        <CardTitle>Critical Equipment Requiring Attention</CardTitle>
      </div>
    </CardHeader>
    <CardContent>
      <div className="space-y-2" data-testid="list-critical-equipment">
        {criticalEquipment.map(eq => (
          <div
            key={eq.id}
            className="flex items-center justify-between p-3 border rounded
            data-testid={`item-critical-equipment-${eq.id}`}
          >
            <div>

```

```

        <p className="font-medium" data-testid={`text-equipment-name-
            {eq.name ?? eq.id}
        </p>
        <div className="flex items-center gap-2 mt-0.5">
            <p className="text-sm text-muted-foreground" data-testid={`
                Health: {eq.healthIndex}% · Failure Risk: {eq.failureRisk
            </p>
            {/* Rec #10 – show data age on risk scores */}
            <DataFreshnessBadge timestamp={eq.lastTelemetryAt} />
        </div>
        </div>
        <Badge variant="destructive" data-testid={`badge-critical-${eq.
        </div>
        )}}
    </div>
</CardContent>
</Card>
)}

{/* — Rec #4 – Three-state anomaly acknowledgement ————— */}
<CollapsibleSection
    title="Recent Anomalies"
    badge={
        newAnomalies.length > 0
        ? `${newAnomalies.length} new${watchingAnomalies.length > 0 ? ` · ${w
        : watchingAnomalies.length > 0
        ? `${watchingAnomalies.length} watching`
        : undefined
    }
    summary={
        newAnomalies.length > 0
        ? `${newAnomalies.length} anomalies require review`
        : 'No unreviewed anomalies'
    }
>
    {anomaliesWithStatus.length === 0 ? (
        <p className="text-sm text-muted-foreground">
            No anomalies.{acknowledgedCount > 0 && ` ${acknowledgedCount} acknowl
        </p>
    ) : (
        <div className="space-y-2">
            {/* New – require action */}
            {newAnomalies.length > 0 && (
                <div>
                    <p className="text-xs font-medium text-muted-foreground mb-1.5 fl
                        <AlertTriangle className="h-3 w-3 text-red-500" />
                    New ({newAnomalies.length})

```

```

    </p>
    <div className="space-y-2">
      {newAnomalies.slice(0, 8).map((a, idx) => (
        <AnomalyRow
          key={a.id ?? idx}
          anomaly={a}
          idx={idx}
          ackStatus="unacknowledged"
          onWatch={handleWatch}
          onAcknowledge={handleAcknowledge}
        />
      ))}
    </div>
  </div>
)}

{/* Watching – operator is aware, still monitoring */}
{watchingAnomalies.length > 0 && (
  <div className="mt-3">
    <p className="text-xs font-medium text-muted-foreground mb-1.5 fl
      <Eye className="h-3 w-3 text-amber-500" />
      Watching ({watchingAnomalies.length})
      <span className="font-normal opacity-70">– monitoring, will esc
    </p>
    <div className="space-y-2">
      {watchingAnomalies.map((a, idx) => (
        <AnomalyRow
          key={a.id ?? idx}
          anomaly={a}
          idx={idx}
          ackStatus="watching"
          onWatch={handleWatch}
          onAcknowledge={handleAcknowledge}
        />
      ))}
    </div>
  </div>
)}

{/* Acknowledged count – shown as quiet footer, not full list */}
{acknowledgedCount > 0 && (
  <p className="text-xs text-muted-foreground pt-2">
    {acknowledgedCount} acknowledged this session
  </p>
)}
</div>
)}

```

```

</CollapsibleSection>

{/* Equipment health trends */}
{(equipmentHealthTrends ?? []).length > 0 && (
  <CollapsibleSection
    title="Equipment Health Trends"
    summary={`Showing trends for ${Math.min(3, equipmentHealthTrends.length
  >
    <div className="space-y-6">
      {equipmentHealthTrends.slice(0, 3).map(eq => (
        <div key={eq.equipmentId} className="space-y-2" data-testid={`item-
          <div className="flex items-center justify-between">
            <h4 className="text-sm font-medium" data-testid={`text-trend-na
              {eq.name}
            </h4>
            <div className="flex items-center gap-2">
              {/* Rec #10 – freshness alongside health badge */}
              <DataFreshnessBadge timestamp={eq.lastTelemetryAt} />
              <Badge
                variant={eq.currentHealth < 50 ? 'destructive' : 'default'}
                data-testid={`badge-trend-health-${eq.equipmentId}`}
              >
                {eq.currentHealth}% Health
              </Badge>
            </div>
          </div>
        <div className="h-48" data-testid={`chart-health-trend-${eq.equip
          <ResponsiveContainer width="100%" height="100%">
            <LineChart data={eq.trendData}>
              <CartesianGrid strokeDasharray="3 3" />
              <XAxis dataKey="timestamp" tick={{ fontSize: 12 }} />
              <YAxis domain={[0, 100]} tick={{ fontSize: 12 }} />
              <Tooltip />
              <Line type="monotone" dataKey="health" stroke="#2563eb" str
            </LineChart>
          </ResponsiveContainer>
        </div>
      </div>
    )})
  </div>
</CollapsibleSection>
)}

{/* Active telemetry streams */}
<div ref={telemetrySectionRef}>
  <CollapsibleSection title="Active Telemetry Streams" summary={`${streams.
    <div className="space-y-4">

```

```

<div className="flex flex-wrap items-center gap-3">
  <Select
    value={selectedVessel}
    onChange={v => { setSelectedVessel(v); setSelectedEquipment(
  >
    <SelectTrigger className="w-[180px]" data-testid="select-vessel">
      <SelectValue placeholder="All Vessels" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="all">All Vessels</SelectItem>
      {vessels.map(v => <SelectItem key={v.id} value={v.id}>{v.name}<
    </SelectContent>
  </Select>

  <Select value={selectedEquipment} onChange={setSelectedEquipme
    <SelectTrigger className="w-[180px]" data-testid="select-equipmen
      <SelectValue placeholder="All Equipment" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="all">All Equipment</SelectItem>
      {filteredEquipment.map(e => <SelectItem key={e.id} value={e.id}
    </SelectContent>
  </Select>

  <Button size="sm" variant="outline" onClick={() => refetchStreams()}
    <RefreshCw className="h-4 w-4 mr-1" />
    Refresh
  </Button>

  {streams.filter(s => s.hasAnomaly).length > 0 && (
    <Badge variant="destructive" data-testid="badge-anomaly-count">
      {streams.filter(s => s.hasAnomaly).length} anomalies
    </Badge>
  )}
</div>

{streamsLoading ? (
  <div className="text-sm text-muted-foreground">Loading telemetry st
) : streams.length === 0 ? (
  <div className="text-sm text-muted-foreground">
    No telemetry streams available. Check sensor configuration.
  </div>
) : (
  <>
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 ga
      {streams.slice(0, 12).map(stream => (
        <SensorSparklineChart

```

```

        key={`${stream.equipmentId}-${stream.sensorType}`}
        sensorType={stream.sensorType}
        equipmentId={stream.equipmentId}
        currentValue={stream.currentValue}
        unit={stream.unit}
        status={stream.status}
        hasAnomaly={stream.hasAnomaly}
        anomalyZScore={stream.anomalyZScore}
        anomalyTimestamp={stream.anomalyTimestamp}
        data={stream.data}
        lastUpdate={stream.lastUpdate}
        onViewDetails={() => handleViewDetails(stream.equipmentId,
        />
    )}}
</div>
{streams.length > 12 && (
    <p className="text-xs text-muted-foreground text-center pt-1">
        Showing 12 of {streams.length} streams. Filter by vessel or e
    </p>
    )}
</>
    )}
</div>
</CollapsibleSection>
</div>
</div>
);
}

```